TESTING NEURAL NETWORKS ON MOBILE DEVICES

Hung Manh Nguyen

Bachelor's thesis
Faculty of Information Technology
Czech Technical University in Prague
Department of Cybernetics
Study program: Informatics
Specialisation: Artificial intelligence
Supervisor: Ing. Jan Čech, Ph.D.
April 14, 2026

Replace the contents of this file with official assignment.
Místo tohoto souboru sem patří list se zadáním závěrečné práce.

Czech Technical University in Prague
Faculty of Information Technology
© 2026 Hung Manh Nguyen. All rights reserved.
This thesis is school work as defined by Copyright Act of the Czech Republic. It has been submitted at Czech Technical University in Prague, Faculty of Information Technology. The thesis is protected by the Copyright Act and its use, with the exception of free statutory licenses and beyond the scope of the authorizations specified in the Declaration, requires the consent of the author.

Citation of this thesis: Nguyen Hung Manh. *Testing Neural Networks on Mobile Devices.* Bachelor's thesis. Czech Technical University in Prague, Faculty of Information Technology, 2026.

I would like to thank my supervisor Ing. Jan Čech, Ph.D. for his guidance, feedback, and support throughout this thesis.
I also thank my family and friends for their encouragement.

Declaration

TODO: Insert one of the approved declaration texts provided by FIT CTU and fill in the required information.

In Praze on April 14, 2026

Abstract

This thesis focuses on the theoretical foundations and benchmarking methodology for mobile face-processing neural networks on Android and iOS devices. It discusses mobile hardware constraints, efficient model design, quantization principles, and the ExecuTorch deployment workflow. The work defines a reproducible protocol for measuring end-to-end latency, throughput, and facial landmark accuracy on AFW, including data logging and validity controls for cross-platform evaluation.

Keywords mobile inference, face detection, facial landmarks, benchmarking, quantization, ExecuTorch, Core ML, XNNPACK

Abstrakt

Práce se věnuje teoretickým základům a metodice benchmarkingu neuronových sítí pro zpracování obličeje na mobilních zařízeních (Android, iOS). Popisuje omezení mobilního hardwaru, principy efektivních architektur a kvantizace, a detailně rozebírá deployment workflow frameworku ExecuTorch. Součástí je návrh reprodukovatelné metodiky pro měření end-to-end latence, propustnosti a přesnosti odhadu obličejových bodů na datasetu AFW.

Klíčová slova mobilní inference, detekce obličeje, obličejové body, benchmark, kvantizace, ExecuTorch, Core ML, XNNPACK

Contents

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Problem statement . . . 1
1.3 Research objectives . . . 2
1.4 Main contributions . . . 2
1.5 Thesis structure . . . 2

2 Theoretical background **3**
2.1 On-device inference in mobile environments . . . 3
2.2 Hardware and system constraints . . . 3
2.2.1 CPU, GPU, and NPU roles . . . 3
2.2.2 Power and thermal behavior . . . 4
2.3 Efficient neural networks for mobile vision . . . 4
2.4 Model compression and quantization . . . 4
2.5 Face detection and landmark estimation . . . 4
2.5.1 Task definitions . . . 4
2.5.2 Representative models . . . 4
2.5.3 Datasets and annotation uncertainty . . . 5
2.6 Benchmarking principles . . . 5

3 ExecuTorch for mobile deployment **6**
3.1 Why ExecuTorch . . . 6
3.2 Conceptual architecture . . . 6
3.3 Export and lowering workflow . . . 6
3.4 Delegation and partitioning . . . 7
3.5 Runtime design and implications for benchmarking . . . 7
3.6 Quantization in ExecuTorch . . . 7
3.7 Practical limitations . . . 8

4 Benchmark methodology **9**
4.1 Methodological goals . . . 9
4.2 Evaluated pipeline . . . 9
4.3 Model families and configuration space . . . 10
4.4 iOS and Android implementation parity . . . 10
4.5 Dataset and inputs . . . 10
4.6 Metrics . . . 11

4.6.1 Latency and throughput 11
4.6.2 Landmark accuracy 11
4.6.3 Optional energy metrics 11
4.7 Measurement protocol 11
4.8 Run artifacts and offline analysis 12
4.9 Threats to validity 12
4.10 Ethical and privacy considerations 12

5 Conclusion **13**

A Reproducibility **14**
A.1 Generate filtered combined CSV (iPhone 14 + Redmi Note 8 Pro, BS <= 8) 14
A.2 Generate curated thesis charts (per-device + combined) 14
A.3 Optional summary tables 14

Bibliography **16**

Contents of the attachment **20**

List of Figures

List of Tables

List of abbreviations

AFW	Annotated Faces in the Wild (dataset)
AUC	Area Under the Curve
CPU	Central Processing Unit
Core ML	Apple on-device machine learning framework
E2E	End-to-end
FPS	Frames per second
INT8	8-bit integer quantization
IoU	Intersection over Union
NME	Normalized Mean Error
NNAPI	Android Neural Networks API
NPU	Neural Processing Unit
PFLD	Practical Facial Landmark Detector
XNNPACK	Optimized CPU backend used by ExecuTorch

Chapter 1
Introduction

This thesis addresses the engineering problem of deploying and evaluating modern face-processing neural networks directly on mobile devices. The work focuses on the theoretical foundations and experimental methodology required for trustworthy benchmarking of face detection and facial landmark estimation on Android and iOS using ExecuTorch and backend delegates.

1.1 Motivation

On-device inference has become a key requirement for many user-facing applications, because it improves privacy, reduces round-trip latency, and allows offline operation. At the same time, mobile deployment is constrained by strict limits on compute throughput, memory bandwidth, power, and thermal budget [1, 2]. In practice, this means that achieving good model accuracy is not sufficient; the full end-to-end system must also be efficient and stable under realistic operating conditions.

Recent community benchmarks confirm that measured latency strongly depends on hardware configuration, runtime backend, numerical precision, and measurement protocol [3, 2]. Therefore, reliable conclusions require not only model comparison but also careful benchmark design and reproducible data processing.

1.2 Problem statement

The core problem of this thesis is to design a methodologically sound framework for evaluating mobile face-processing pipelines implemented with ExecuTorch. The framework must support equivalent iOS and Android applications, consistent measurement conditions, and post-hoc validation of quality metrics.

The benchmarked workload is an end-to-end two-stage pipeline:

1. face detection in an input image,

2. facial landmark estimation on detected face crops.

The thesis intentionally focuses on theoretical and methodological parts. Numerical benchmark outcomes are out of scope of this text.

1.3 Research objectives

Theoretical and engineering objectives of the thesis are:

- summarize the main concepts of on-device inference relevant to mobile face-processing;
- describe ExecuTorch architecture and deployment workflow for mobile targets [4, 5];
- define a reproducible benchmark protocol for latency, throughput, and landmark accuracy;
- identify validity threats (thermal throttling, backend asymmetry, unsupported operators, logging artifacts) and propose mitigations.

1.4 Main contributions

The thesis contributes:

- a structured theoretical overview connecting mobile hardware constraints, model optimization, and runtime system design;
- an ExecuTorch-focused deployment explanation covering export, lowering, delegation, and runtime execution [6, 7, 8];
- a practical methodology for fair cross-platform benchmarking of detector–landmark pipelines;
- a reproducibility-oriented data handling workflow suitable for independent audit and future extension.

1.5 Thesis structure

Chapter 2 introduces theoretical background: mobile inference constraints, efficient model design, quantization, and face-processing tasks. Chapter 3 explains ExecuTorch architecture and backend delegation for mobile deployment. Chapter 4 defines the benchmark methodology, metrics, protocol, and validity controls. The final chapter concludes the thesis.

Chapter 2

Theoretical background

2.1 On-device inference in mobile environments

Mobile AI systems are typically deployed on heterogeneous Systems-on-Chip (SoCs), where CPU, GPU, and dedicated NPUs coexist [9, 1]. Compared to server inference, mobile workloads operate under tighter thermal and power limits and often share resources with interactive foreground tasks.

From a systems perspective, latency depends on several layers:

- model architecture (operator types, tensor shapes, parameter count),
- runtime backend (kernel library, operator fusion, threading policy),
- hardware mapping (CPU/GPU/NPU dispatch),
- memory subsystem behavior (cache locality, bandwidth pressure),
- thermal governor and dynamic voltage/frequency scaling (DVFS).

As a consequence, the same neural network can show substantially different performance across devices and software stacks [2, 3].

2.2 Hardware and system constraints

2.2.1 CPU, GPU, and NPU roles

CPUs provide broad operator coverage and predictable behavior, but may not offer the highest throughput for dense tensor computations. Mobile GPUs can improve throughput for selected workloads, while NPUs may deliver superior energy-efficiency when supported operators map well to accelerator kernels [9, 1]. In real deployments, unsupported subgraphs frequently fall back to CPU, introducing synchronization and memory transfer overhead.

2.2.2 Power and thermal behavior

Peak performance is often not sustainable in handheld devices. Prolonged inference can trigger thermal throttling, reducing frequency and increasing latency variance. Benchmark methodology must therefore control ambient conditions, warm-up state, and per-run timing windows [2].

2.3 Efficient neural networks for mobile vision

The shift from server-centric CNNs to mobile-friendly architectures was enabled by operator-level efficiency ideas such as depthwise separable convolution, inverted residual blocks, and channel shuffle [10, 11, 12]. SqueezeNet demonstrated that competitive accuracy can be achieved with significantly fewer parameters [13].

For mobile face-processing, both detector and landmark models must be efficient. The detector controls how many candidate faces are forwarded to the second stage, while the landmark model dominates per-face regression cost.

2.4 Model compression and quantization

Compression and quantization reduce memory footprint and often improve runtime performance. Classical compression includes pruning and weight sharing [14]. In practical mobile deployment, integer quantization is especially important because INT8 kernels can be faster and more energy-efficient than FP32 on supported hardware [15].

Post-training quantization (PTQ) is attractive for deployment pipelines because it does not require full retraining. However, sensitivity varies across models and layers; accuracy degradation can occur without representative calibration and proper backend support [16].

2.5 Face detection and landmark estimation

2.5.1 Task definitions

Face detection localizes faces with bounding boxes. Landmark estimation predicts keypoint coordinates (e.g., eye corners, nose tip, mouth corners), usually on a cropped face image. These tasks are strongly coupled in end-to-end systems: detection quality and crop geometry directly influence landmark quality.

2.5.2 Representative models

RetinaFace is a strong one-stage detector with high robustness under challenging conditions [17]. FaceBoxes is a lightweight detector explicitly designed for CPU real-time operation [18].

For landmarks, PFLD emphasizes compactness and speed [19]. MobileFaceNet provides a lightweight backbone originally developed for mobile face recognition and often reused for landmark regression [20]. PIPNet combines heatmap-style reasoning with low-resolution efficient heads for practical landmark localization [21].

2.5.3 Datasets and annotation uncertainty

In this thesis, AFW annotations are used for wild-condition landmark evaluation, originating from the face detection and pose estimation work of Zhu and Ramanan [22]. As with most in-the-wild datasets, annotation noise, partial occlusion, and pose diversity should be treated as intrinsic uncertainty rather than pure model error.

2.6 Benchmarking principles

Prior benchmarking studies stress that meaningful comparisons require controlled experimental factors: fixed run rules, warm-up handling, hardware affinity, consistent data preprocessing, and transparent logging [3, 2]. Consequently, this thesis adopts a protocol-first approach: theory and methodology are defined before any result interpretation.

Chapter 3

ExecuTorch for mobile deployment

3.1 Why ExecuTorch

ExecuTorch is a PyTorch edge deployment stack designed for efficient on-device inference across mobile and embedded targets [4, 5]. Its central design principle is *ahead-of-time* (AOT) preparation: expensive transformation and planning steps are performed before deployment, so runtime execution remains lightweight.

3.2 Conceptual architecture

ExecuTorch organizes model deployment into three phases [4, 5]:

1. export a PyTorch model to graph form,
2. lower and optimize the graph for edge execution,
3. execute the serialized program with a small C++ runtime.

The terminology of ATen/Core ATen/Edge/Backend dialects defines progressive lowering from general PyTorch representation toward target-specific execution [23].

3.3 Export and lowering workflow

The practical workflow consists of:

1. model export using `torch.export`,
2. backend-aware lowering via `to_edge_transform_and_lower`,

3. emission of a serialized `.pte` artifact.

This process supports dynamic shape constraints and optional separation of program and tensor data into `.pte` and `.ptd` files [6, 24, 25, 26].

3.4 Delegation and partitioning

ExecuTorch uses delegates to offload supported subgraphs to backend-specific implementations [8]. A partitioner tags lowerable graph regions; unsupported regions remain in portable execution. This mixed execution model improves portability but can create overhead from frequent backend boundaries.

For this thesis, two backends are central:

- **XNNPACK**: optimized CPU backend with broad mobile support, including quantized paths [27, 28];
- **Core ML delegate**: Apple backend path that can dispatch to CPU, GPU, or ANE depending on model and platform capabilities [29, 30].

3.5 Runtime design and implications for benchmarking

The runtime loads `.pte` programs, resolves operator kernels or delegate entry points, and executes a preplanned instruction sequence [7]. Because planning is mostly AOT, runtime overhead is low and easier to reason about than dynamic graph execution.

For benchmarking, this has three practical implications:

- model initialization and first-run costs must be separated from steady-state latency;
- backend registration/linking correctness is mandatory, otherwise runs silently degrade to slower fallback paths;
- exported artifact provenance (backend, precision, shape constraints) must be tracked in run metadata.

3.6 Quantization in ExecuTorch

ExecuTorch integrates quantization workflows through backend-specific configurations, commonly using PT2E/torchao-based preparation and conversion [31, 32]. Since quantization support differs across delegates and operators, each quantized artifact must be validated for both numerical behavior and actual backend coverage.

3.7 Practical limitations

In real projects, deployment quality is affected by operator support matrices, toolchain version compatibility, and backend-specific converter behavior. These issues are not edge cases; they are a normal part of mobile ML engineering and must be documented alongside benchmark methodology.

Chapter 4

Benchmark methodology

4.1 Methodological goals

The benchmark design in this thesis follows two goals:

1. produce comparable latency/throughput/accuracy measurements across iOS and Android,
2. preserve raw artifacts needed for independent recomputation and audit.

The approach extends prior benchmarking ideas from mobile inference literature [2, 3] and adapts them to face-processing pipelines.

4.2 Evaluated pipeline

Each benchmark run executes an end-to-end pipeline:

1. detector preprocessing,
2. detector inference,
3. detector postprocessing (box decoding and filtering),
4. face crop generation,
5. landmark model preprocessing,
6. landmark inference,
7. landmark postprocessing.

End-to-end timing includes all stages above, not only neural forward passes. This reflects application-level latency perceived by users.

4.3 Model families and configuration space

The methodology supports multiple detector/landmark pairs and backend configurations. Typical factors include:

- detector choice (e.g., RetinaFace vs FaceBoxes),
- landmark architecture (e.g., PFLD, MobileFaceNet, PIPNet),
- precision variant (FP32, FP16 where available, INT8),
- backend delegate (XNNPACK, Core ML, or portable fallback),
- resolved batch size.

A complete run record stores these factors explicitly, so later analysis does not depend on file naming conventions alone.

4.4 iOS and Android implementation parity

Because platform asymmetries can bias conclusions, iOS and Android benchmark applications are designed to be feature-equivalent at the pipeline level. Equivalent behavior includes:

- identical preprocessing parameters,
- identical postprocessing logic and thresholds,
- consistent metric collection and export schema,
- consistent handling of warm-up and measured iterations.

This principle follows the project guideline that iOS and Android applications should remain equivalent in features and purpose.

4.5 Dataset and inputs

AFW-based landmark annotations are used for accuracy-oriented offline recomputation [22]. Input assets and ordering are versioned as part of run metadata. For each run, used-input logs are exported to avoid ambiguity caused by dynamic file-system ordering.

4.6 Metrics

4.6.1 Latency and throughput

Let T_{run} be measured end-to-end runtime for N processed frames. The average throughput is

$$\text{FPS} = \frac{N}{T_{\text{run}}}. \tag{4.1}$$

Per-frame latency is measured in milliseconds and summarized by robust statistics (median and selected percentiles), not only arithmetic means.

4.6.2 Landmark accuracy

For N faces and K landmarks, normalized mean error (NME) is defined as

$$\text{NME} = \frac{1}{N}\sum_{n=1}^{N}\frac{1}{K}\sum_{k=1}^{K}\frac{\|\hat{\mathbf{p}}_{n,k} - \mathbf{p}_{n,k}\|_2}{d_n}, \tag{4.2}$$

where $\hat{\mathbf{p}}_{n,k}$ is predicted landmark position, $\mathbf{p}_{n,k}$ is ground truth, and d_n is the normalization distance (e.g., inter-ocular distance).

4.6.3 Optional energy metrics

If platform tooling allows reliable collection, energy-related signals can be logged as supplementary indicators. In this thesis, latency and accuracy metrics remain primary.

4.7 Measurement protocol

To reduce noise and measurement bias, each benchmark condition follows a fixed protocol:

- dedicated warm-up iterations before timed measurement,
- fixed number of measured iterations,
- stable device state (charging policy, background app control, thermal preconditioning),
- separate logging for initialization, inference, and optional postprocessing stages.

Runs that violate schema, contain missing key payloads, or fail consistency checks are flagged for exclusion from aggregate statistics.

4.8 Run artifacts and offline analysis

Every run exports a machine-readable summary plus raw samples. Offline analysis scripts recompute aggregate statistics and, when possible, recompute accuracy from per-image predictions instead of trusting prefilled summary fields. This post-hoc recomputation improves robustness against stale metadata and partial export errors.

4.9 Threats to validity

Main threats and mitigations include:

- **Thermal drift**: controlled warm-up and repeated runs;
- **Backend asymmetry**: shared-configuration comparisons for cross-device claims;
- **Unsupported operators**: explicit reporting of delegate coverage and fallback paths;
- **Data leakage or ordering artifacts**: deterministic input lists and exported input manifests;
- **Metric inconsistency**: central offline recomputation of statistics.

4.10 Ethical and privacy considerations

The thesis evaluates technical performance of face-processing models and does not develop identity recognition functionality. Nevertheless, any deployment of face analytics should follow legal and ethical constraints for biometric data, including transparent purpose limitation, secure storage, and user consent where required.

Chapter 5
Conclusion

This thesis presents the theoretical and methodological basis for benchmarking mobile face-processing inference with ExecuTorch. The text first establishes mobile inference constraints, then connects them to efficient model design, quantization, and runtime backend behavior. It further explains ExecuTorch's export-to-runtime path, delegate mechanism, and artifact formats relevant to practical deployment.

A reproducible benchmark methodology is defined for end-to-end detector plus landmark pipelines on iOS and Android, with explicit metric definitions, protocol controls, and validity safeguards. The methodology is designed to support fair cross-platform comparison and independent recomputation from raw artifacts.

The resulting framework provides a solid base for empirical evaluation and future extension, including additional devices, backends, and optimization strategies. Numerical evaluation outcomes are intentionally outside the scope of this text.

Appendix A
Reproducibility

This appendix lists the commands used to regenerate the CSV data and the charts used in the thesis.

A.1 Generate filtered combined CSV (iPhone 14 + Redmi Note 8 Pro, BS <= 8)

From the repository root:

```
./.venv/bin/python benchmarks/analysis/plot_runs_dashboard.py \
  --mobile-root benchmarks/results/mobile \
  --pc-root benchmarks/results/pc/runs \
  --devices "iphone 14,xiaomi redmi note 8 pro" \
  --max-resolved-batch 8 \
  --output-csv benchmarks/results/combined/combined_runs_phone_compare_bs8.csv \
  --output-dir benchmarks/results/combined/charts_phone_compare_bs8 \
  --focus-device "iphone 14"
```

A.2 Generate curated thesis charts (per-device + combined)

```
./.venv/bin/python benchmarks/analysis/plot_phone_pair_thesis.py \
  --input-csv benchmarks/results/combined/combined_runs_phone_compare_bs8.csv \
  --output-dir thesis/images/benchmarks \
  --max-batch 8
```

A.3 Optional summary tables

```
./.venv/bin/python benchmarks/analysis/summarize_benchmarks.py \
```

```
--input benchmarks/results/combined/combined_runs_phone_compare_bs8.csv \
--output-dir benchmarks/results/combined/tables_phone_compare_bs8
```

Bibliography

1. WANG, Siqi; PATHANIA, Anuj; MITRA, Tulika. Neural Network Inference on Mobile SoCs. *IEEE Design & Test*. 2020, vol. 37, no. 5, pp. 50–57. Available from DOI: 10.1109/MDAT.2020.2968258.
2. LI, Zhuojin; PAOLIERI, Marco; GOLUBCHIK, Leana. A Benchmark for ML Inference Latency on Mobile Devices. In: *Proceedings of the 7th International Workshop on Edge Systems, Analytics and Networking (EdgeSys '24)*. 2024. Available from DOI: 10.1145/3642968.3654818.
3. REDDI, Vijay Janapa; KANTER, David; MATTSON, Peter; DUKE, Jared; NGUYEN, Thai; CHUKKA, Ramesh; SHIRING, Ken; TAN, Koan-Sin; CHARLEBOIS, Mark; CHOU, William, et al. MLPerf Mobile Inference Benchmark: An Industry-Standard Open-Source Machine Learning Benchmark for On-Device AI. *Proceedings of Machine Learning and Systems*. 2022, vol. 4, pp. 352–369. Available also from: https://proceedings.mlsys.org/paper_files/paper/2022/hash/a2b2702ea7e682c5ea2c20e8f71efb0c-Abstract.html.
4. PYTORCH FOUNDATION. *How ExecuTorch Works* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/intro-how-it-works.html.
5. PYTORCH FOUNDATION. *Architecture and Components — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/getting-started-architecture.html.
6. PYTORCH FOUNDATION. *Model Export and Lowering — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/using-executorch-export.html.
7. PYTORCH FOUNDATION. *ExecuTorch Runtime Overview* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/runtime-overview.html.

8. PYTORCH FOUNDATION. *Understanding Backends and Delegates — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: `https://pytorch.org/executorch/stable/compiler-delegate-and-partitioner.html`.

9. LEE, Juhyun; CHIRKOV, Nikolay; IGNASHEVA, Ekaterina; PISARCHYK, Yury; SHIEH, Mogan; RICCARDI, Fabio; SAROKIN, Raman; KULIK, Andrei; GRUNDMANN, Matthias. On-Device Neural Net Inference with Mobile GPUs. *arXiv preprint arXiv:1907.01989*. 2019. Available from DOI: `10.48550/arXiv.1907.01989`.

10. HOWARD, Andrew G.; ZHU, Menglong; CHEN, Bo; KALENICHENKO, Dmitry; WANG, Weijun; WEYAND, Tobias; ANDREETTO, Marco; ADAM, Hartwig. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv preprint arXiv:1704.04861*. 2017. Available from DOI: `10.48550/arXiv.1704.04861`.

11. SANDLER, Mark; HOWARD, Andrew; ZHU, Menglong; ZHMOGINOV, Andrey; CHEN, Liang-Chieh. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 4510–4520. Available from DOI: `10.1109/CVPR.2018.00474`.

12. ZHANG, Xiangyu; ZHOU, Xinyu; LIN, Mengxiao; SUN, Jian. ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 6848–6856. Available from DOI: `10.1109/CVPR.2018.00716`.

13. IANDOLA, Forrest N.; MOSKEWICZ, Matthew W.; ASHRAF, Khalid; HAN, Song; DALLY, William J.; KEUTZER, Kurt. SqueezeNet: AlexNet-Level Accuracy with 50x Fewer Parameters and <1MB Model Size. *arXiv preprint arXiv:1602.07360*. 2016. Available from DOI: `10.48550/arXiv.1602.07360`.

14. HAN, Song; MAO, Huizi; DALLY, William J. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. *arXiv preprint arXiv:1510.00149*. 2016. Available from DOI: `10.48550/arXiv.1510.00149`.

15. JACOB, Benoit; KLIGYS, Skirmantas; CHEN, Bo; ZHU, Menglong; TANG, Matthew; HOWARD, Andrew; ADAM, Hartwig; KALENICHENKO, Dmitry. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 2704–2713. Available from DOI: `10.1109/CVPR.2018.00286`.

16. KRISHNAMOORTHI, Raghuraman. Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper. *arXiv preprint arXiv:1806.08342*. 2018. Available from DOI: `10.48550/arXiv.1806.08342`.

17. DENG, Jiankang; GUO, Jia; VERVERAS, Evangelos; KOTSIA, Irene; ZAFEIRIOU, Stefanos. RetinaFace: Single-Stage Dense Face Localisation in the Wild. *arXiv preprint arXiv:1905.00641*. 2020. Available from DOI: 10.48550/arXiv.1905.00641.

18. ZHANG, Shifeng; ZHU, Xiangyu; LEI, Zhen; SHI, Hailin; WANG, Xiaobo; LI, Stan Z. FaceBoxes: A CPU Real-Time Face Detector with High Accuracy. In: *2017 IEEE International Joint Conference on Biometrics (IJCB)*. 2017, pp. 1–9. Available from DOI: 10.1109/BTAS.2017.8272675.

19. GUO, Xiaojie; LI, Siyuan; YU, Jinke; ZHANG, Jiawan; MA, Jiayi; MA, Lin; LIU, Wei; LING, Haibin. PFLD: A Practical Facial Landmark Detector. *arXiv preprint arXiv:1902.10859*. 2019. Available from DOI: 10.48550/arXiv.1902.10859.

20. CHEN, Sheng; LIU, Yang; GAO, Xiang; HAN, Zhen. MobileFaceNets: Efficient CNNs for Accurate Real-Time Face Verification on Mobile Devices. In: *Biometric Recognition (CCBR 2018)*. Springer, 2018, vol. 10996, pp. 428–438. Lecture Notes in Computer Science. Available from DOI: 10.1007/978-3-319-97909-0_46.

21. JIN, Haibo; LIAO, Shengcai; SHAO, Ling. Pixel-in-Pixel Net: Towards Efficient Facial Landmark Detection in the Wild. *International Journal of Computer Vision*. 2021, vol. 129, no. 12, pp. 3174–3194. Available from DOI: 10.1007/s11263-021-01521-4.

22. ZHU, Xiangxin; RAMANAN, Deva. Face Detection, Pose Estimation, and Landmark Localization in the Wild. In: *2012 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2012, pp. 2879–2886. Available from DOI: 10.1109/CVPR.2012.6248014.

23. PYTORCH FOUNDATION. *Concepts — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/concepts.html.

24. PYTORCH FOUNDATION. *.pte file format — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/pte-file-format.html.

25. PYTORCH FOUNDATION. *.ptd file format — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/ptd-file-format.html.

26. PYTORCH FOUNDATION. *torch.export User Guide* [online]. 2026. [visited on 2026-04-14]. Available from: https://docs.pytorch.org/docs/stable/user_guide/torch_compiler/export.html.

27. PYTORCH FOUNDATION. *XNNPACK Backend — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/backends/xnnpack/xnnpack-overview.html.

28. GOOGLE. *XNNPACK: High-efficiency Neural Network Operators* [online]. 2026. [visited on 2026-04-14]. Available from: https://github.com/google/XNNPACK.

29. PYTORCH FOUNDATION. *Core ML Backend — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/backends/coreml/coreml-overview.html.

30. APPLE. *Core ML* [online]. 2026. [visited on 2026-04-14]. Available from: https://developer.apple.com/documentation/coreml.

31. PYTORCH FOUNDATION. *Quantization Overview — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/quantization-overview.html.

32. PYTORCH AO CONTRIBUTORS. *PT2E Quantization Tutorials and Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://docs.pytorch.org/ao/main/tutorials_source/pt2e_quant_ptq.html.

Contents of the attachment

The electronic attachment contains the source code of the benchmark applications, benchmarking scripts, collected run exports and the thesis sources.

```
/
├── android/ ........................ Android benchmark app source code
├── ios/ ............................... iOS benchmark app source code
├── benchmarks/ ......... Benchmark harness, analysis scripts, and results
│   ├── analysis/ .... JSON aggregation, charting and table export scripts
│   └── results/ ....... Collected exported runs + generated charts/tables
├── models/ .......... Detector and landmark model files used by the apps
└── thesis/ ................................ LaTeX sources of the thesis
    ├── ctufit-thesis.tex ...................... Main thesis entry point
    └── text/ ........................................... Chapter sources
```